

Inspecto — 5-Hour Hackathon Build Guide

Building an interactive multilingual factory-visit report assistant with Claude Code · Trishala Adwaine Suresh · BearingPoint

What you're demoing: upload a factory field-note document (any language) plus visit photos/video. Inspecto translates and structures the notes, runs computer-vision analysis on the images/video frames to flag safety, compliance and quality issues, and outputs one standardized bilingual visit report — on screen and as a downloadable Word file.

Architecture (buildable solo in Claude Code)

Layer	Tool / Library
Interface	Streamlit — fastest way to get file uploaders + a live results page with zero front-end code
Text + translation	python-docx to read field notes → Claude (Messages API) to translate + extract structured JSON (site, date, observations, issues)
Computer vision	Claude vision (image blocks in the Messages API) scores each photo/frame against a fixed checklist (PPE, hazards, equipment condition, cleanliness)
Video handling	Claude has no native video input — extract 1 frame every 2–3 sec with OpenCV/ffmpeg, then run them through the same vision step as photos
Output	Merge text + vision JSON → render as tabs in Streamlit and generate a formatted Word report with python-docx

Hour-by-Hour Plan

Time	Do this — tell Claude Code:
Hr 1 Setup	Scaffold a Streamlit app "Inspecto" with uploaders for a .docx field-notes file, multiple images, and one video. Add a requirements.txt with anthropic, streamlit, python-docx, opencv-python, pillow. Read my ANTHROPIC_API_KEY from env.
Hr 2 Text	Write a function that extracts text from the uploaded .docx with python-docx, sends it to Claude with a prompt to translate to English and return strict JSON (site, date, inspector, observations[], issues[]), and parses the JSON reliably.
Hr 3 Vision	Write a function that extracts 1 frame every 2 seconds from the video with OpenCV, base64-encodes each photo/frame, and sends each to Claude vision with a fixed rubric prompt (PPE compliance, visible hazards, equipment condition, cleanliness, 1-5 severity). Return JSON per image and aggregate flags.
Hr 4 Report	Merge the text JSON and vision JSON into one report object. In Streamlit show 3 tabs — Summary, Translated Notes, Visual Findings (thumbnail + flag + severity). Add a "Download Word Report" button that builds a formatted .docx with python-docx from the same object.
Hr 5 Polish	Add one sample field-note doc (non-English) and 4-5 sample photos as defaults so the demo runs with one click. Add a loading spinner, basic error handling for missing files, and BearingPoint pink branding to the Streamlit theme. Rehearse the 3-minute run-through below.

3-Minute Demo Script

- Open with the problem: factory visits today = informal notes, no structure, no translation.
- Upload the sample field notes (in French/Spanish) + sample photos/video — click Generate.
- Show the Translated Notes tab: raw foreign-language text next to the structured English summary.
- Show Visual Findings: each photo/frame with its auto-flagged issue and severity score.
- Click Download Word Report — show the polished, ready-to-share output.
- Close with scale-up: same pipeline reuses for ESG audits, store visits, compliance checks.

Gotchas & Quick Wins

- Claude accepts images (JPEG/PNG/GIF/WebP) but not raw video — frame extraction is mandatory, not optional.
- Force structured output: ask for "valid JSON only, no prose" and wrap the parse in a try/except with one retry.
- Keep the vision rubric fixed and identical across photos/frames — this is what makes the report standardized.
- Quick win if time allows: color-code severity (green/amber/red) in both the Streamlit view and the Word report.
- Stretch goal: let the report template be configurable per client (ESG checklist vs. safety checklist).